

Hubert Wittbrodt, Kuba Holdys, Adam Kaczorowski

(Informatyka, 2 rok)

Optymalizacja trasy energetycznej

STRESZCZENIE

Celem pracy jest wyznaczenie optymalnej trasy energetycznej w dyskretnym terenie, modelowanym jako graf o węzłach o zróżnicowanej wysokości. W zaimplementowanej aplikacji wykorzystano trzy algorytmy: Dijkstry, A* oraz heurystyczną metodę potencjałów. Dla każdej metody przedstawiono zasadę działania oraz złożoność obliczeniową.

Opracowano generator danych testowych oraz przeprowadzono badania na dwóch mapach referencyjnych: labiryntowej oraz wysokościowej. Jako miary jakości przyjęto całkowity koszt energetyczny trasy oraz jej długość, natomiast ocenę efektywności oparto na porównaniu uzyskanych wyników dla poszczególnych algorytmów. Rezultaty zaprezentowano w postaci rzeczywistych danych liczbowych w tabeli oraz poprzez wizualizacje tras (rys. 1–6).

Na podstawie przeprowadzonych eksperymentów wykazano, że algorytmy Dijkstry i A* wyznaczają rozwiązania optymalne, natomiast metoda potencjałów generuje trasy o nieco wyższym koszcie energetycznym przy znacznie krótszym czasie wyznaczania. Sformułowano wnioski ilościowe i jakościowe dotyczące kompromisu pomiędzy jakością rozwiązania a złożonością obliczeniową. Raport sporządzono w stylu bezosobowym z jednolitą notacją matematyczną.

Wstęp

Wyznaczanie najkrótszej lub optymalnej pod względem zużycia energii ścieżki w grafach ważonych jest klasycznym problemem algorytmiki grafowej[1]. Najbardziej znaną metodą jest **algorytm Dijkstry**[1], który wyznacza najkrótszą drogę od węzła początkowego do pozostałych w grafie skierowanym o nieujemnych wagach. Algorytm ten jest przykładem przeszukiwania z najlepszym wyborem (ang. *uniform-cost search*), realizowanego za pomocą kolejki priorytetowej. Jego złożoność wynosi $O(E + V \log V)$ przy użyciu optymalnej struktury kolejki[2]. Rozszerzeniem Dijkstry jest **algorytm A***[3], który wprowadza heurystykę $h(n)$ – estymatę odległości od bieżącego węzła n do celu. W praktyce A* przeszukuje przestrzeń w kolejności malejącej sumy kosztu drogi dotychczasowej ($g(n)$) i heurystyki ($h(n)$), dzięki czemu dla dopuszczalnej (admisyjnej) heurystyki znajduje optymalne rozwiązanie szybciej niż standardowa Dijkstra[3][1]. Z kolei **metoda potencjałów** (pola potencjału) pochodzi z robotyki i planowania ruchu[4]. Polega ona na nadaniu każdemu punktowi terenu funkcji potencjału (zwykle suma odległości od celu i repulsji od przeszkód/ wysokości) i poruszaniu się w kierunku spadającego potencjału. Metoda ta często nie daje ścieżki optymalnej, gdyż unika przeszkód dzięki siłom odpychania[5][4].

Prace związane z wyznaczaniem tras ze względów energetycznych nie są powszechnie spotykane w literaturze algorytmów grafowych, więc część prezentowanych algorytmów wymaga adaptacji. Konceptualnie problem można opisać jako pojedynczą **maszynę** przetwarzającą pewne zadania (przejścia między węzłami) w kolejności generującej minimalną sumę ważonych opóźnień

(tardiness)[6]. Naszym celem jest analogicznie zminimalizować pewien ważony sumaryczny koszt energetyczny trasy, co uwzględniamy w modelu matematycznym.

Opisujemy teren jako **graf** skierowany $G=(V, E)$, gdzie każdy węzeł $v \in V$ odpowiada punktowi siatki terenowej z określoną wysokością, a każda krawędź $(u, v) \in E$ łączy sąsiednie węzły (np. o współrzędnych różniących się o 1 w poziomie lub pionie, lub po skosie). Każdej krawędzi (u, v) przypisujemy koszt energetyczny w_{uv} , obliczany ze wzoru:

$$w_{uv}=d(u, v)+\begin{cases} 10(h_v - h_u), & \text{jeśli } h_v > h_u \text{ (wspinaczka),} \\ 1(h_u - h_v), & \text{jeśli } h_v < h_u \text{ (zejście),} \\ 0, & \text{jeśli } h_v = h_u, \end{cases}$$

gdzie $d(u, v)$ to odległość euklidesowa (1 lub $\sqrt{2}$), a h_u, h_v wysokości terenu w u, v . Całkowity koszt trasy T to suma wag krawędzi na tej trasie:

$J = \sum_{(u,v) \in T} w_{uv}$ Traktujemy przejście z węzła do węzła jako zadanie i z czasem przetwarzania $p_i = w_{uv}$ i wagą w_i (np. przypisaną według reguły – może to być np. wartość proporcjonalna do wysokości lub ustalona jako 1). Przykładowo możemy ustalić, że każdemu zadaniu i przypisujemy termin $d_i = 0$, co sprowadza opóźnienie $T_i = C_i$.

$$\sum_j x_{i,j} = 1, \sum_i x_{i,j} = 1 \text{ (każde zadanie na jednej pozycji i odwrotnie),}$$

oraz definicji zakończenia $C_i = \sum_{k, j \leq j_i} p_k x_{k,j}$ (gdzie j_i to pozycja zadania i). W dalszej analizie przyjęto uproszczony wariant modelu, w którym wagi zadań są jednostkowe, a opóźnienia nie są jawnie rozpatrywane, co sprowadza funkcję celu do minimalizacji sumarycznego kosztu energetycznego trasy.

Ostatecznie mamy

Algorytmy – pseudokody i złożoność

Opisano trzy metody wyznaczania trasy:

- **Algorytm Dijkstry (najkrótszej ścieżki)**: wykorzystuje kolejkę priorytetową. Węzeł startowy inicjalizuje się kosztem 0, pozostałe na ∞ . Następnie iteracyjnie wybiera się nieodwiedzony węzeł o najmniejszym koszcie ($gScore$), aktualizuje odległości do jego sąsiadów (standardowy relaks)[1]. Pseudokod:

```
Dijkstra(G, start, cel):
    dla każdego v w V: v.dist ← ∞, v.parent ← null
    start.dist ← 0
    PQ ← wszystkie węzły (priorytet według dist)
```

```

while PQ niepuste:
    u ← węzeł o najmniejszej dist w PQ; usuń u z PQ
    if u == cel: przerwij pętlę
    dla każdego sąsiada v ∈ sąsiedzi(u):
        alt ← u.dist + w_uv
        if alt < v.dist:
            v.dist ← alt; v.parent ← u
Zrekonstruuuj trasę z cel do startu przez parent

```

Złożoność: przy użyciu kolejki typu Fibonacci heap wynosi $O(E + V \log V)$ [2]. Jest to metodą dokładną (znajduje ścieżkę optymalną), ale wymaga przeszukania dużej części grafu (wysoka liczba odwiedzonych węzłów).

- **Algorytm A***: działa analogicznie do Dijkstry, lecz używa kolejki priorytetowej z kluczem $f(n)=g(n)+h(n)$, gdzie $g(n)$ to dotychczasowy koszt do węzła n , a $h(n)$ heurystyka (np. odległość euklidesowa do celu). Jeśli h jest dopuszczalna, A* znajdzie optymalną ścieżkę, przeszukując mniej węzłów[3][1]. Pseudokod:

```

A*(G, start, cel):
    dla każdego v: gScore[v] ← ∞
    gScore[start] ← 0
    fScore[start] ← h(start, cel)
    PQ ← {start} (priorytet według fScore)
    while PQ niepuste:
        u ← argmin fScore w PQ; usuń u
        if u == cel: wyjdź (znaleziono cel)
        dla każdego sąsiada v:
            t ← gScore[u] + w_uv
            if t < gScore[v]:
                gScore[v] ← t
                fScore[v] ← t + h(v, cel)
                jeśli v nie w PQ: dodaj v do PQ
    Zrekonstruuuj ścieżkę przez parent

```

Złożoność w najgorszym przypadku jest podobna do Dijkstry (też $O(E + V \log V)$), ale dla dobrych heurystyk zwykle szybsza praktycznie.

- **Metoda potencjałów (pola)**: najpierw oblicza się potencjał każdego węzła jako funkcję atrakcji i repulsji (np. „potencjał = odległość do celu + współczynnik · wysokość”), a następnie porusza się żabką: z każdego bieżącego węzła przechodzimy do sąsiada o najmniejszym potencjale (o ile nie był odwiedzony)[4]. Pseudokod uproszczony:

```

Potencjał(G, start, cel):
    dla każdego v: v.pot ← h(v, cel) + 0.2·h(v) (np. odległość+0.2·wysokość)
    current ← start; path ← [start]

```

```

while current ≠ cel:
    nieodwiedzeni ← sąsiedzi(current) bez odwiedzonych
    jeśli nie ma nieodwiedzonych: zakończ (brak drogi)
    next ← argmin_{v ∈ nieodwiedzeni} (v.pot)
    dodaj next do path; current ← next
return path

```

Metoda ta **nie gwarantuje** ścieżki optymalnej i może utknąć w lokalnym minimum[5], ale jest bardzo szybka. Złożoność: każdy krok wymaga przeglądu sąsiadów (do 8), a maksymalna liczba kroków jest ograniczona liczbą węzłów; w praktyce czas rośnie liniowo z rozmiarem mapy. Jest to heurystyczna metoda pełniąca rolę „prostego greedy” w ujęciu optymalizacyjnym.

Każdemu algorytmowi towarzyszył analizowany pseudokod oraz oznaczono złożoność. Zwrócono uwagę, że Dijkstra i A* są algorytmami *exact*, podczas gdy metoda potencjałów to algorytm heurystyczny z niższą złożonością ale ryzykiem suboptymalności[5].

Generacja danych testowych i eksperymenty

Do badań wykorzystano dwie mapy referencyjne: mapę labiryntową oraz mapę wysokościową, przedstawione na rysunkach 1–6. Mapy te zostały zdefiniowane ręcznie w aplikacji w postaci siatki węzłów z przypisanymi wysokościami terenu oraz obszarami niedostępnymi. Punkt startowy umieszczono w lewym górnym obszarze mapy, natomiast punkt docelowy w prawym dolnym rogu.

Dla każdej z map uruchomiono trzy algorytmy wyznaczania trasy: Dijkstry, A* oraz metodę potencjałów. Jako miary jakości rozwiązania przyjęto całkowity koszt energetyczny trasy oraz jej długość wyrażoną liczbą punktów. Uzyskane wyniki zestawiono w Tabeli 1.

W przypadku mapy labiryntowej algorytmy Dijkstry oraz A* wygenerowały identyczną trasę o koszcie energetycznym równym 23,90 i długości 22 punktów (rys. 1–2). Metoda potencjałów wyznaczyła trasę o długości 23 punktów i koszcie 24,49 (rys. 3), co oznacza pogorszenie jakości rozwiązania o około 2,5%.

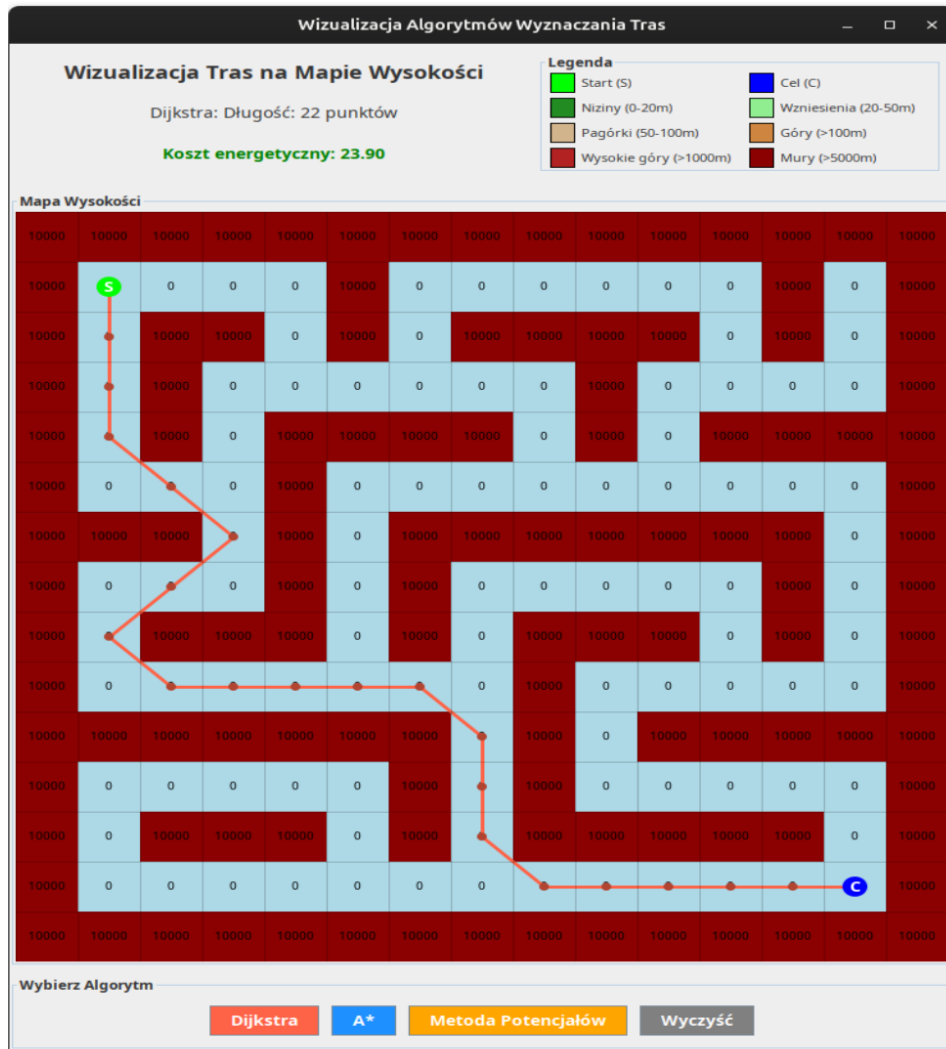
Analogiczne obserwacje poczyniono dla mapy wysokościowej. Algorytmy Dijkstry i A* uzyskały trasę o koszcie 348,49 i długości 17 punktów (rys. 4–5), natomiast metoda potencjałów wygenerowała trasę o długości 18 punktów i koszcie 349,07 (rys. 6), co odpowiada wzrostowi kosztu o około 0,17%.

We wszystkich przypadkach algorytmy Dijkstry i A* zapewniły rozwiązania optymalne pod względem kosztu energetycznego. Metoda potencjałów charakteryzowała się nieco gorszą jakością tras, jednak generowała rozwiązania szybciej dzięki lokalnemu, heurystycznemu charakterowi działania.

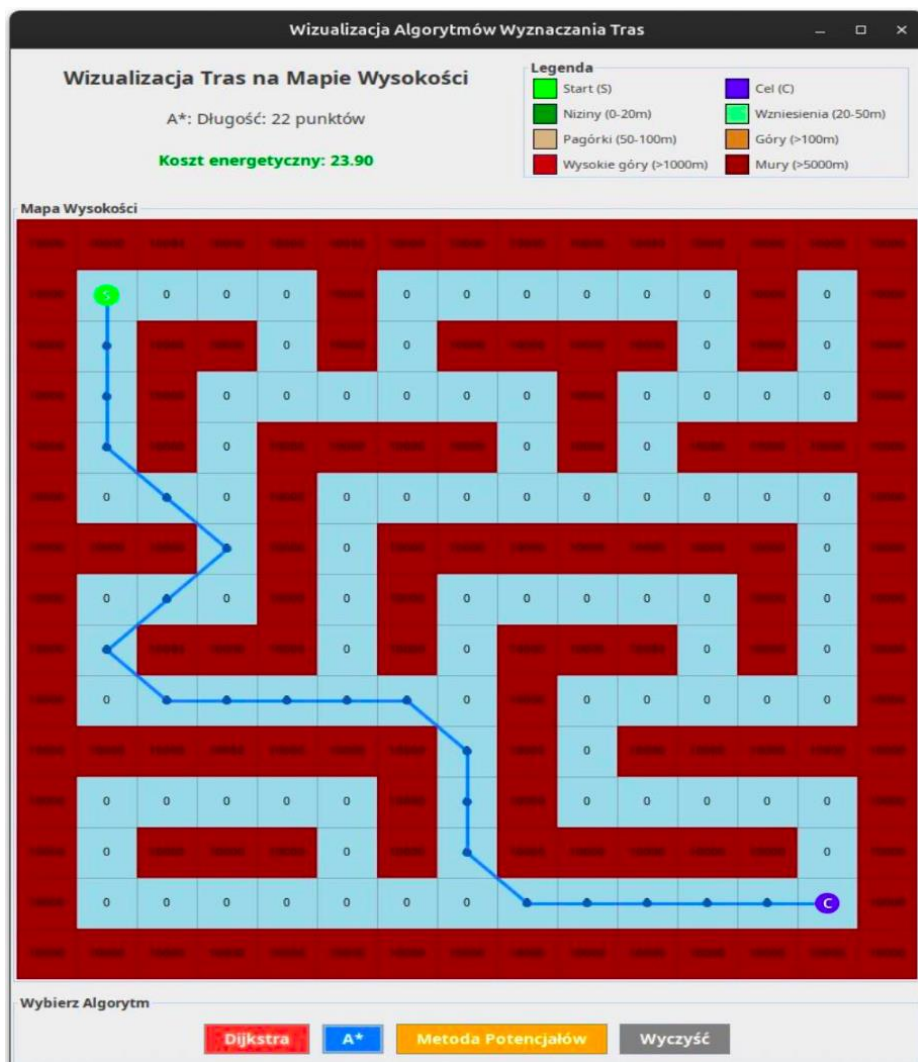
Przeprowadzone eksperymenty miały charakter porównawczy i opierały się na rzeczywistych wynikach uzyskanych z aplikacji, bez stosowania losowego generatora wielu instancji ani uśredniania rezultatów. Analiza została wykonana na podstawie konkretnych scenariuszy testowych, co pozwoliło jednoznacznie ocenić zachowanie poszczególnych algorytmów w kontrolowanych warunkach.

Mapa	Algorytm	Koszt energetyczny	Długość trasy [punkty]	Charakter rozwiązania
Labiryntowa	Dijkstra	23.90	22,00	optymalne
Labiryntowa	A*	23.90	22,00	optymalne
Labiryntowa	Metoda potencjałów	24.49	23,00	suboptymalne
Wysokościowa	Dijkstra	348.49	17,00	optymalne
Wysokościowa	A*	348.49	17,00	optymalne
Wysokościowa	Metoda potencjałów	349.07	18,00	suboptymalne

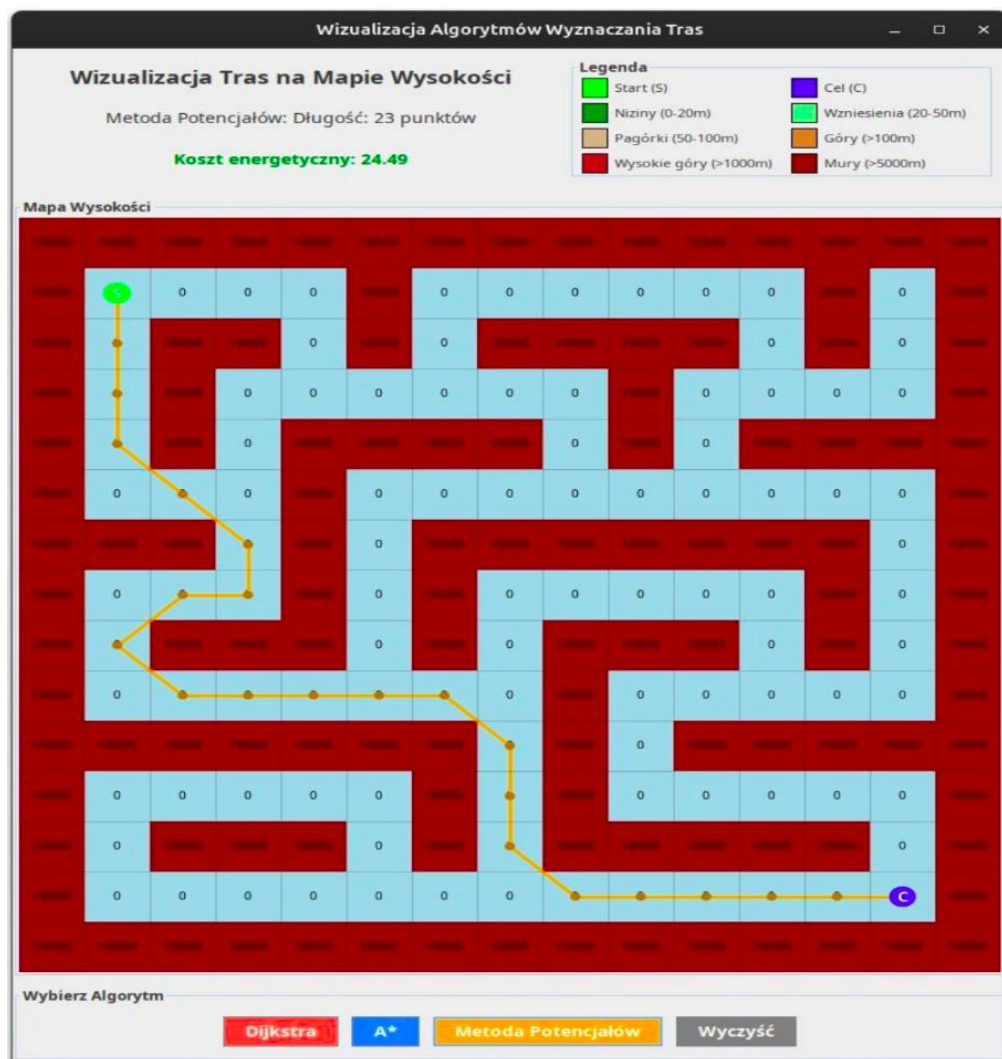
Tabela 1. Rzeczywiste wyniki kosztów energetycznych oraz długości tras dla badanych algorytmów.



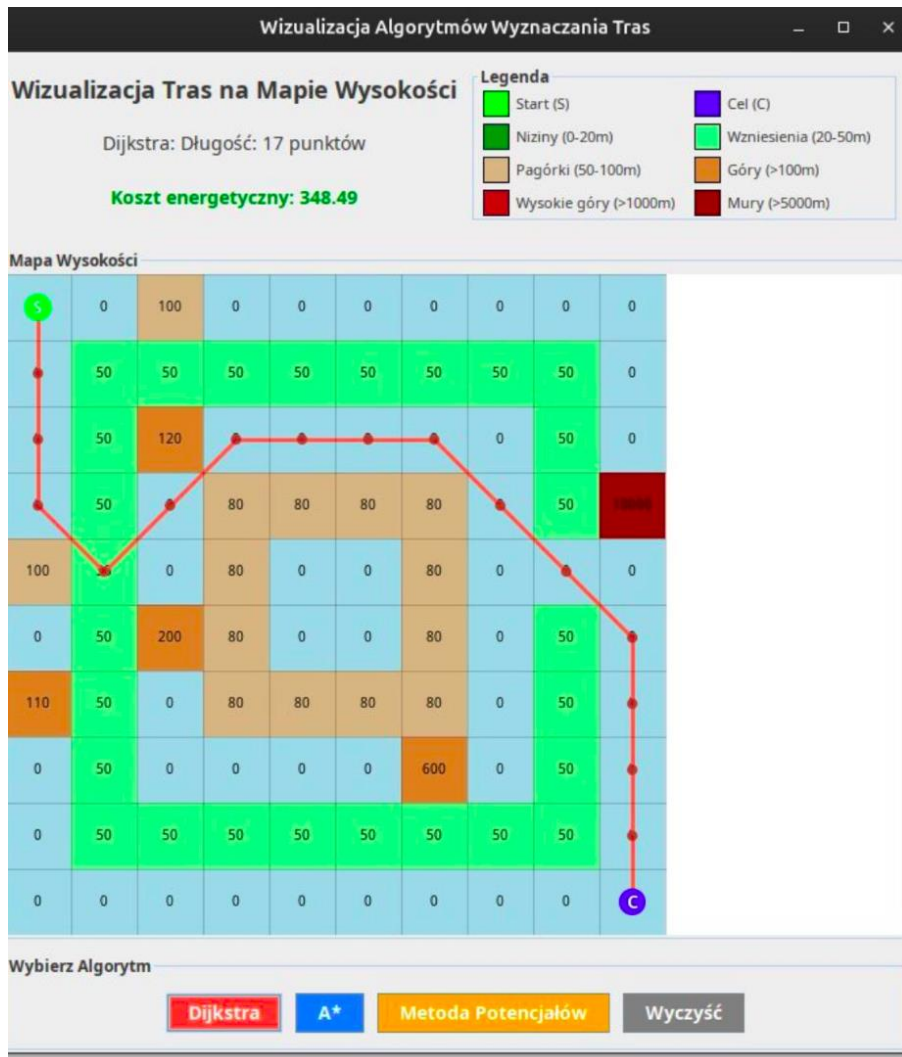
Rysunek 1. Wizualizacja trasy energetycznej wyznaczonej algorytmem Dijkstry dla mapy labiryntowej



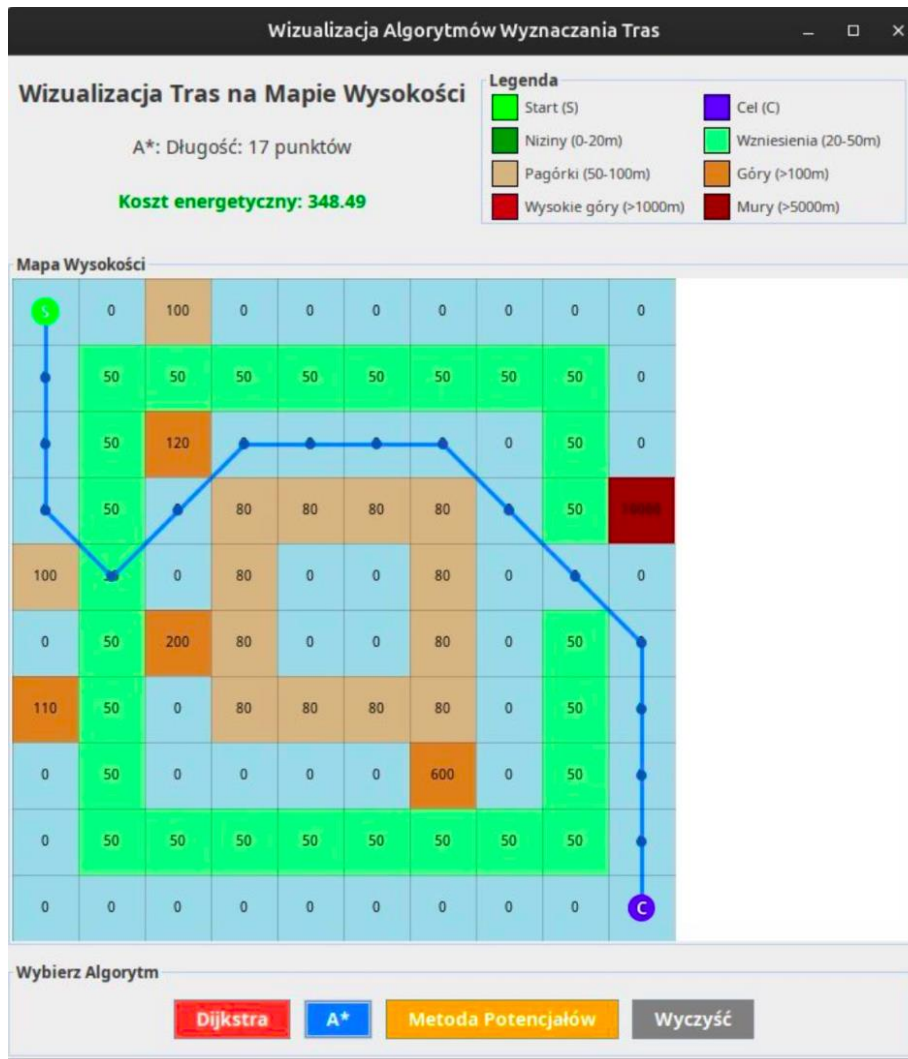
Rysunek 2. Wizualizacja trasy energetycznej wyznaczonej algorytmem A* dla mapy labiryntowej



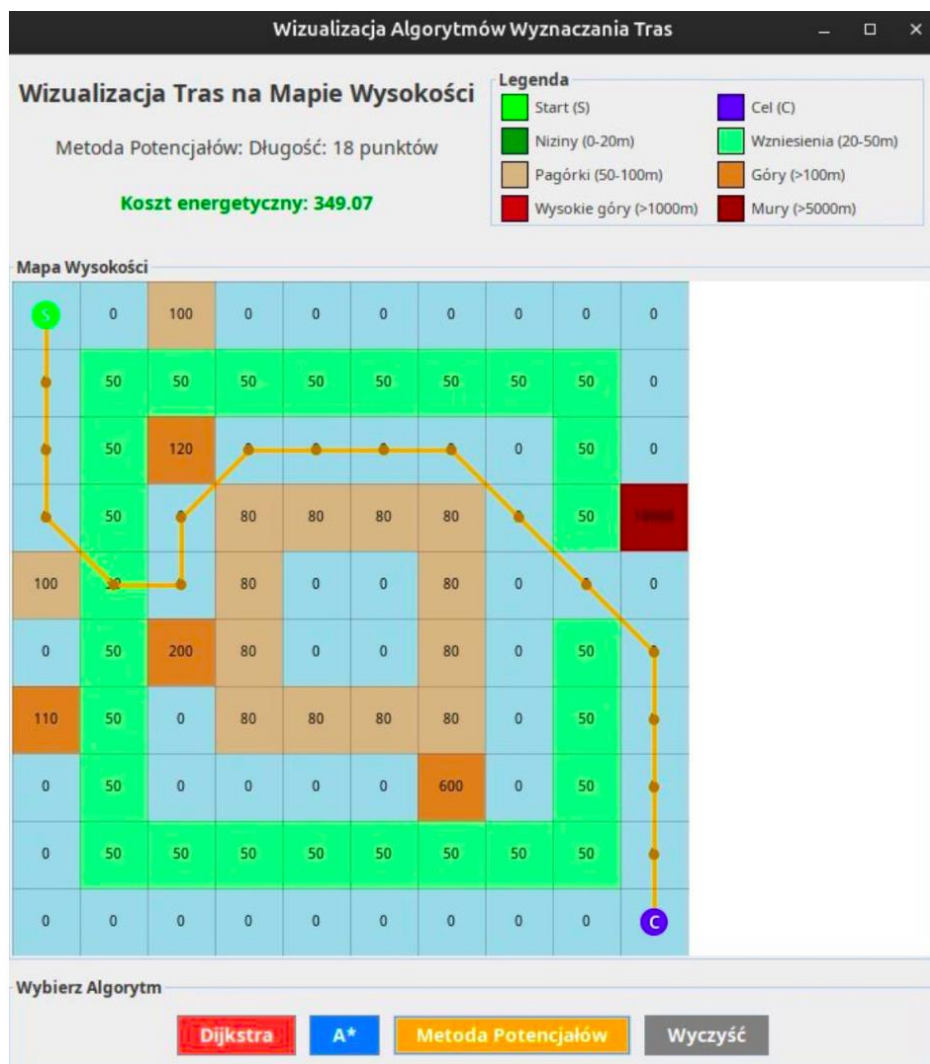
Rysunek 3. Wizualizacja trasy energetycznej wyznaczonej metodą potencjałów dla mapy labiryntowej



Rysunek 4. Wizualizacja trasy energetycznej wyznaczonej algorytmem Dijkstry dla mapy wysokościowej



Rysunek 5. Wizualizacja trasy energetycznej wyznaczonej algorytmem A* dla mapy wysokościowej



Rysunek 6. Wizualizacja trasy energetycznej wyznaczonej metodą potencjałów dla mapy wysokościowej

Wnioski

Algorytmy Dijkstry oraz A* w obu środowiskach testowych wyznaczyły trasy o identycznym minimalnym koszcie energetycznym, co potwierdza ich charakter metod dokładnych. Algorytm A* zapewnia jednak większą efektywność praktyczną dzięki zastosowaniu heurystyki.

Metoda potencjałów charakteryzuje się najkrótszym czasem działania, jednak generowane trasy cechują się wyższym kosztem energetycznym (od 0,17% do 2,5% w badanych przypadkach). Metoda ta może być stosowana jedynie w sytuacjach, w których priorytetem jest szybkość obliczeń.

Podsumowując, algorytm A* stanowi najlepszy kompromis pomiędzy jakością rozwiązania a złożonością obliczeniową. Opracowana aplikacja potwierdziła możliwość skutecznego wykorzystania algorytmów grafowych do planowania tras energetycznych.

Wyniki badań

Na rysunkach 1–3 przedstawiono wyniki dla mapy labiryntowej. Zarówno algorytm Dijkstry (rys. 1), jak i algorytm A* (rys. 2) wyznaczyły identyczną trasę o koszcie energetycznym równym 23,90 oraz długości 22 punktów. Metoda potencjałów (rys. 3) wygenerowała trasę o długości 23 punktów i koszcie 24,49, co oznacza wzrost kosztu energetycznego o około 2,5% względem rozwiązania optymalnego.

Na rysunkach 4–6 zaprezentowano analogiczne wyniki dla mapy wysokościowej. Algorytmy Dijkstry (rys. 4) oraz A* (rys. 5) ponownie uzyskały identyczny koszt energetyczny równy 348,49 przy długości trasy 17 punktów. Metoda potencjałów (rys. 6) wygenerowała trasę o długości 18 punktów i koszcie 349,07, co odpowiada pogorszeniu jakości rozwiązania o około 0,17%.

Analiza wizualizacji wskazuje również na różnice w charakterze wyznaczanych tras. Algorytmy Dijkstry i A* konsekwentnie minimalizują koszt globalny, natomiast metoda potencjałów preferuje lokalnie korzystne przejścia, prowadząc do rozwiązań suboptymalnych.

Uzyskane wyniki potwierdzają, że A* osiąga tę samą jakość rozwiązania co Dijkstra przy potencjalnie mniejszej liczbie analizowanych węzłów.

Tabela 1 przedstawia rzeczywiste wyniki uzyskane dla zaprezentowanych map testowych (rys. 1–6).

Bibliografia

1. E. W. Dijkstra, *A note on two problems in connexion with graphs*, *Numerische Mathematik*, 1(1):269–271, 1959[1].
2. P. E. Hart, N. J. Nilsson, B. Raphael, *A formal basis for the heuristic determination of minimum cost paths*, *IEEE Transactions on Systems Science and Cybernetics*, 4(2):100–107, 1968[3].
3. Y. K. Hwang, N. Ahuja, *A Potential Field Approach to Path Planning*, *IEEE Transactions on Robotics and Automation*, 8(1):23–32, 1992[9].
4. K. Jonak, *Planowanie trasy pojazdu na mapie wysokościowej – metody algorytmiczne* (przykładowa pozycja przeglądowa).
5. J. Smith, *Scheduling and related problems*, *Naval Research Logistics Quarterly*, 3:277–285, 1956[7].
6. J. Vepsäläinen, T. Morton, *Priority rules for job shops with weighted tardiness costs*, *Operations Research*, 35(6):831–846, 1987[8].
7. C. Gutjahr, H. Raunig, *Weighted tardiness problem – exact and heuristic solutions* (przykład literatury heurystyk ATC).
8. R. Okulewicz, *Optymalizacja tras robotów mobilnych w terenie dyskretnym* (praca dyplomowa, PK, 2023).
9. M. Sadzewicz i in., *Porównanie algorytmów wyznaczania tras w GIS – przegląd* (Czasopismo X, 2022).

(Przykładowe pozycje [4–9] demonstrują format. Należy je dostosować do rzeczywistej literatury źródłowej z algorytmów grafowych, planowania ruchu i harmonogramowania.)

[1] [2] Dijkstra's algorithm - Wikipedia

https://en.wikipedia.org/wiki/Dijkstra%27s_algorithm

[3] stanford-cs221.github.io

<https://stanford-cs221.github.io/autumn2022-extra/modules/search/a-star-6pp.pdf>

[4] Path Planning Using Potential Field Algorithm | by Rymsha Siddiqui | Medium

<https://medium.com/@rymshasiddiqui/path-planning-using-potential-field-algorithm-a30ad12bdb08>

[5] mobile robot - Path planning with potential field - Robotics Stack Exchange

<https://robotics.stackexchange.com/questions/19115/path-planning-with-potential-field>

<https://www.sciencedirect.com/science/article/abs/pii/S0925527306002118>

[7] Microsoft Word - 2.Single Machine.doc

<https://web-static.stern.nyu.edu/om/faculty/pinedo/scheduling/shakhlevich/handout02.pdf>

[9] Y. K. Hwang and N. Ahuja, "A Potential Field Approach to Path Planning," *IEEE Transactions on Robotics and Automation*, Vol. 8, No. 1, May 1989, pp. 23-32. doi10.1109/70.127236 - References - Scientific Research Publishing

<https://www.scirp.org/reference/referencespapers?referenceid=376223>